

RELATÓRIO DO PROJETO: SISTEMA DE ATENDIMENTO PARA BARBEARIA (BARBERPOO)

1. Título

Sistema de Gerenciamento e Agendamento de Atendimento para Barbearia

2. Descrição

O projeto consiste no desenvolvimento de um software em linguagem **Python** voltado para a automação e o gerenciamento do fluxo de atendimento de uma barbearia. O sistema permite o cadastro de clientes, o registro de profissionais (barbeiros), a consulta de serviços disponíveis (como cortes de cabelo e barba) e a realização de agendamentos vinculando cliente, profissional, serviço e data/horário de atendimento através de uma interface de terminal (CLI).

3. Justificativa

A escolha do tema "Barbearia" justifica-se pela alta aplicabilidade comercial de sistemas voltados para o setor de serviços e pela excelente aderência ao paradigma de **Programação Orientada a Objetos (POO)**. O cenário real de uma barbearia mapeia perfeitamente entidades do mundo real (clientes, profissionais, serviços prestados e horários de atendimento), permitindo aplicar de forma natural e clara conceitos avançados de modelagem de software, reutilização de código e organização estrutural.

4. Funcionalidade

- **Cadastro de Clientes:** Permite registrar novos clientes informando nome, telefone e status de fidelidade (VIP).
- **Cadastro de Barbeiros:** Permite registrar os profissionais da barbearia com nome, telefone e suas respectivas especialidades.
- **Catálogo de Serviços:** Exibe a lista de serviços oferecidos pela barbearia (ex: Corte, Barba, Combo), informando o preço e a duração estimada.
- **Realização de Agendamentos:** Associa um cliente cadastrado a um barbeiro e a um serviço escolhido, registrando a data e o horário exatos do atendimento.
- **Listagem de atendimentos:** Exibe um resumo completo de todos os agendamentos realizados no sistema.

5. Temas Abordados (Conceitos de POO Aplicados)

- **Abstração:** Utilização da classe abstrata Pessoa (através do módulo abc) para definir uma estrutura genérica e obrigatória que as subclasses devem seguir.

- **Herança:** A classe base abstrata Pessoa foi herdada pelas classes filhas Cliente e Barbeiro, evitando duplicação de código para atributos comuns como nome e telefone.
- **Encapsulamento:** Proteção dos dados internos das classes utilizando atributos protegidos (ex: `_nome`) e o uso de propriedades (`@property`) para acesso controlado.
- **Polimorfismo:** Implementação do método abstrato `exibir_dados()` na classe mãe, que foi sobrescrito de maneira específica tanto na classe Cliente quanto na classe Barbeiro.
- **Associação / Composição:** A classe Agendamento unifica instâncias das classes Cliente, Barbeiro e Servico, demonstrando a colaboração entre diferentes objetos do sistema.

6. Tecnologias e Bibliotecas

- **Linguagem de Programação:** Python (versão 3.x).
- **Biblioteca Nativa:** `datetime` (utilizada para manipulação, formatação e validação de datas e horários dos agendamentos).
- **Controle de Versão:** Git e GitHub (para organização do repositório em pastas `src` e `docs`).

7. Distribuição de Tarefas

- **Emanuel de Jesus Sousa:** Modelagem de classes base, implementação da Abstração, Herança e Encapsulamento (`models.py`).
- **Samuel Machado Magalhães:** Implementação das classes de negócio (`Servico`, `Agendamento`) e integração com manipulação de datas (`datetime`).
- **Italo Santana de Oliveira:** Desenvolvimento da interface de menu interativo no terminal (`main.py`), testes do sistema, organização do repositório no GitHub e formatação do relatório final.